

Tasarım Raporu

Bu raporda, tasarım desenlerinin neden seçildiğine ve her birinin hangi problemi çözdüğüne dair teknik açıklama sunulacaktır.

Süreç	Karşılaşılan Problem	Seçilen Tasarım Deseni	Neden bu desen?
Sipariş aşamaları	Durum geçişleri (Pending, Confirmed, InPreparation, InTransit ve Delivered) karmaşık ve bakımı zor if-else blokları oluştururdu. Her yeni durum, mevcut kodun değiştirilmesini gerektirirdi	State Pattern	Her durum kendi sınıfı içinde kapsüllenmiştir (PendingState, InTransitState vb.). Order sınıfı davranışlarını mevcut duruma devreder ve böylece if-else kullanımına ihtiyaç kalmaz. Yeni bir durum eklemek, mevcut kodu değiştirmeyi gerektirmez
Ödeme yöntemleri	Sistem, Kredi Kartı, Banka Transferi ve ileride Kripto Para gibi ödeme yöntemlerini desteklemek zorundadır. Bir tasarım deseni olmadan, her yeni ödeme yöntemi controller içinde yeni if-else blokları eklenmesini gerektirirdi	Strategy Pattern	IPaymentStrategy arayüzü, sözleşmeyi tanımlar. Her ödeme yöntemi bağımsız bir strateji olarak uygulanır. Controller yalnızca arayüz üzerinden çalışır ve hangi implementasyonu kullandığını bilmez. Yeni ödeme yöntemleri, mevcut koda dokunmadan eklenebilir

İşlem Kayıt Sistemi	Her kritik işlemin (ödeme, stok güncelleme, sipariş oluşturma) kaydedilmesi gerekir. Eğer Logger sınıfından birden fazla örnek (instance) oluşturulsaydı, kayıtlar tutarsız hale gelir ve farklı yerlere dağılırdı.	Singleton Pattern	Logger sınıfı, özel (private) bir yapıcı metoda (constructor) ve uygulama genelinde yalnızca tek bir nesne oluşturulmasını garanti eden statik bir GetInstance() metoduna sahiptir. Sistemdeki tüm sınıflar Logger.GetInstance().Log(...) komutunu kullanır; bu sayede her zaman aynı nesne ve aynı günlük (log) dosyası üzerinden işlem yapılır.
Kargo firmalarıyla entegrasyon	Her kargo firması (Aras, Yurtiçi, GlobalExpress) tamamen farklı metotlara sahip dış bir API sunar. Sistem, bu dış API'lere doğrudan bağımlı olamaz	Adapter Pattern	Her dış API, ICarrier arayüzünü implement eden bir Adapter ile sarılmıştır (ArasAdapter, YurticiAdapter, GlobalExpressAdapter). Sistem yalnızca ICarrier üzerinden iletişim kurar.
Kargo firmalarının oluşturulması	OrderController, kargo firmalarının somut sınıflarını bilmemelidir. Doğrudan örnekleme yapmak (new ArasAdapter() gibi) güçlü bir bağımlılık (tight coupling) oluşturur.	Factory Method Pattern	CarrierFactory sınıfı, kargo firmalarının oluşturulmasını merkezileştirir. Controller, GetCarrier("aras") çağrısını yapar ve hangi somut sınıfın oluşturulduğunu bilmeden bir ICarrier nesnesi alır. Bu yapı, controller'ı değiştirmeden yeni kargo firmalarının eklenmesini kolaylaştırır.
Gönderim için ek hizmetler	Gönderim maliyeti, Sigorta ve Kırılabilir Ürün Koruması gibi isteğe bağlı hizmetleri herhangi bir kombinasyonla içerebilir. Kalıtım (inheritance) kullanmak, çok fazla alt sınıf oluşmasına (subclass explosion) neden olurdu.	Decorator Pattern	InsuranceDecorator ve FragileProtectionDecorator, ICarrier arayüzünü implement eder ve herhangi bir kargo firmasını dinamik olarak sarar. Yeni hizmetler, mevcut sınıflar değiştirilmeden sisteme eklenebilir.

Stok düşük bildirimleri	Bir ürünün stok seviyesi eşik değerinin (threshold) altına düştüğünde, iki farklı birimin farklı yollarla bilgilendirilmesi gerekir: Satın Alma departmanı (e-posta) ve Depo Personeli (dahili bildirim). Bu mantığın doğrudan InventoryService içine dahil edilmesi, Tek Sorumluluk Prensipli'ni ihlal eder	Observer Pattern	IStockObserver arayüzü (interface) sözleşmeyi tanımlar. EmailObserver, AdminEmailBox.txt dosyasına yazarak Satın Alma birimi için e-posta gönderimini simüle eder. InternalNotificationObserver ise bildirimi, Depo Personelinin (WarehouseStaff) panelinde görebilmesi için veri tabanına kaydeder. Product (Ürün), stok azaldığında kayıtlı olan tüm gözlemcileri (observers) otomatik olarak bilgilendirir.